



UFC

UNIVERSIDADE FEDERAL DO CEARÁ

CAMPUS QUIXADÁ

PROGRAMA DE GRADUAÇÃO EM REDES DE COMPUTADORES

MATHEUS HENRIQUE

ANA CLARA

**RELATÓRIO DA ENTREGA DO TRABALHO 3 - Web Services (WS) ou
Application Interface Programming (API)**

QUIXADÁ

Relatório do Projeto: Sistema de Gerenciamento de Peças com API REST

1. Introdução

Este projeto é uma aplicação desenvolvida em Java que possibilita a administração de peças por meio de uma API REST. Ela é estruturada em dois componentes principais: um cliente que se comunica com o servidor para adicionar e buscar peças, e um servidor (PecaServerApi) que controla as peças e disponibiliza endpoints para realizar operações CRUD (Criar, Ler, Atualizar, Excluir).

2. Tecnologias Utilizadas

Java 17: Linguagem de programação utilizada para desenvolver a aplicação.

Spring Boot 3.2.0: Framework para desenvolvimento de aplicações Java, facilitando a criação de APIs REST.

Python 3.x: Linguagem utilizada para o desenvolvimento do segundo cliente, focando em simplicidade e integração rápida.

Node.js (JavaScript): Ambiente de execução utilizado para o terceiro cliente, demonstrando a integração com ecossistemas assíncronos.

Maven: Ferramenta de automação de build utilizada para gerenciar dependências e compilar o projeto

JSON: Formato de dados utilizado para troca de informações entre cliente e servidor.

3.1. Cliente (PecaClientApi.java)

O cliente é responsável por interagir com o usuário e enviar requisições para o servidor. Ele oferece um menu interativo com as seguintes opções:

Adicionar Peça: O usuário insere o nome, código e quantidade da peça, que são enviados para o servidor via POST.

Listar Peças: O cliente faz uma requisição GET para obter a lista de todas as peças cadastradas.

Atualizar Peça: O usuário insere o código da peça e os novos dados (nome e quantidade), que são enviados para o servidor via PUT.

Deletar Peça: O usuário insere o código da peça, que é enviado para o servidor via DELETE.

Sair: Encerra a execução do cliente.

O cliente utiliza a classe HttpClient para enviar requisições HTTP e processar as respostas do servidor.

3.2. Cliente Python (peca_client.py)

O cliente em Python foi implementado utilizando a biblioteca requests. Ele oferece uma interface de linha de comando que permite ao usuário realizar todas as operações CRUD no servidor.

3.3. Cliente JavaScript (cliente.js)

Este cliente foi desenvolvido em Node.js utilizando a biblioteca axios para o gerenciamento de requisições e a readline-sync para a captura de dados via terminal.

3.4. Servidor (PecaServerApi.java)

O servidor é uma aplicação Spring Boot que expõe endpoints REST para gerenciar peças. Ele possui os seguintes endpoints:

GET /pecas: Retorna a lista de todas as peças cadastradas.

POST /pecas: Adiciona uma nova peça à lista.

GET /pecas/{codigo}: Retorna uma peça específica com base no código.

PUT /pecas/{codigo}: Atualiza uma peça existente com base no código.

DELETE /pecas/{codigo}: Remove uma peça com base no código.

GET /pecas/quantidade-total: Retorna a quantidade total de peças cadastradas.

3.5. Modelo de Dados (Peca.java)

A classe Peca representa o modelo de dados utilizado para armazenar informações sobre as peças. Ela contém os seguintes atributos:

nome: Nome da peça.

codigo: Código único da peça.

quantidade: Quantidade disponível da peça.

A classe também inclui métodos getters e setters para acessar e modificar esses atributos.

3.6. Configuração do Projeto (pom.xml)

o projeto é configurado pelo arquivo **pom.xml**, que define as dependências e plugins necessários para a execução do Spring Boot. As principais configurações incluem:

spring-boot-starter-web: Dependência fundamental para a criação de APIs RESTful.

spring-boot-maven-plugin: Plugin responsável por empacotar a aplicação como um arquivo JAR executável.

Java Version: Configurado para a versão 17, garantindo compatibilidade com os recursos modernos da linguagem.

3.7. Configuração de Propriedades (application.properties)

O arquivo application.properties contém configurações para desabilitar a configuração automática do DataSource e do JPA, já que o projeto não utiliza banco de dados.

4. Funcionamento do Projeto

4.1. Execução do Servidor

Para executar o servidor, basta rodar a classe PecaServerApi como uma aplicação Spring Boot. O servidor estará disponível em <http://localhost:8080>.

4.2. Execução dos Clientes

O sistema permite a execução de múltiplos clientes em diferentes linguagens, garantindo a interoperabilidade da API.

Cliente Java: Pode ser executado diretamente a partir da classe PecaClientApi. Ele interage com o usuário via console, utilizando o HttpClient nativo para enviar as requisições ao servidor.

Cliente Python: Deve ser executado via terminal através do comando `python peca_client.py`. Este cliente utiliza a biblioteca requests para realizar a comunicação síncrona com os *endpoints* REST.

Cliente JavaScript (Node.js): Deve ser executado no ambiente Node.js com o comando `node cliente.js`. Ele utiliza a biblioteca axios para chamadas assíncronas e a readline-sync para capturar os dados fornecidos pelo usuário no console.

4.3. Fluxo de Operações

Adicionar Peça: O usuário insere os dados da peça (nome, código e quantidade), que são enviados para o servidor via POST.

Listar Peças: O cliente faz uma requisição GET para obter a lista de todas as peças cadastradas.

Atualizar Peça: O usuário insere o código da peça e os novos dados (nome e quantidade), que são enviados para o servidor via PUT.

Deletar Peça: O usuário insere o código da peça, que é enviado para o servidor via DELETE.

Consultar Quantidade Total: O cliente faz uma requisição GET para obter a quantidade total de peças cadastradas.

5. Como o projeto utiliza uma API REST?

Neste projeto, o servidor (PecaServerApi) expõe *endpoints* RESTful que seguem os princípios da arquitetura REST. Esses *endpoints* permitem que diferentes clientes — implementados em **Java, Python e JavaScript** interajam com o servidor para realizar operações sobre o recurso "peças".

a) Endpoints RESTful

O servidor define rotas que seguem as convenções REST para manipulação de recursos:

- **GET /pecas:** Retorna a lista de todas as peças (leitura).
- **POST /pecas:** Adiciona uma nova peça (criação).
- **PUT /pecas/{codigo}:** Atualiza uma peça existente com base no código (atualização).
- **DELETE /pecas/{codigo}:** Remove uma peça com base no código (exclusão).
- **GET /pecas/{codigo}:** Retorna uma peça específica com base no código (leitura).
- **GET /pecas/quantidade-total:** Retorna a quantidade total de itens no estoque (leitura).

b) Uso de Métodos HTTP e Interoperabilidade

Os clientes utilizam métodos HTTP padronizados para interagir com o servidor:

- **POST:** Utilizado para criação de novos registros.
- **GET:** Utilizado para leitura e recuperação de dados
- **PUT:** Utilizado para atualização de dados existentes.
- **DELETE:** Utilizado para a exclusão de recursos.

Esta padronização permite que o cliente Python (via biblioteca requests) e o cliente JavaScript (via axios) consumam o mesmo serviço Java sem necessidade de adaptações no servidor, cumprindo o requisito de **sistemas distribuídos heterogêneos**.

c) Formato de Dados (JSON)

A troca de informações ocorre exclusivamente via **JSON**, formato independente de linguagem.

Exemplo de Payload enviado pelo cliente (Java, Python ou JS):

```
{  
  "nome": "Parafuso",  
  "codigo": "123",  
  "quantidade": 10  
}
```